False News Detection Using Hybrid Deep Learning and Online Verification

Project Report

1. Introduction

The spread of fake news is a huge problem right now. Information moves so fast that people rarely have time to verify what they are reading, which causes a lot of unnecessary fear and confusion.

For this project, I wanted to create a tool to automatically detect false information. But instead of just trusting a machine learning model—which can easily get confused by brand-new topics—I paired it with live online fact-checking. This hybrid approach makes the system much more practical. The AI handles the initial prediction, and the internet provides the actual proof.

2. Problem Statement

Standard AI models struggle with breaking news because their knowledge is frozen in time. At the same time, expecting everyday people to manually fact-check every headline just isn't realistic. To solve this, I built an automated verification tool that works in three steps:

- **Initial Evaluation:** An AI model makes a baseline prediction on whether a statement seems legitimate.
- **Live Evidence Gathering:** The system actively scours the internet to cross-reference the claim with real-time sources.
- **The Final Verdict:** It combines the AI's initial read with the up-to-the-minute evidence to deliver a definitive answer.

3. Objectives

Here’s exactly what I set out to do with this build:

- Grab the WELFake dataset and train a solid text classifier on it.
- Run some tests to see how good the model actually is at catching fake stuff.
- Wire up a few live APIs (mainly Wikipedia, GNews, and Google Fact Check) so the system isn't just relying on old data.
- Take the model's initial guess, mix it with the live search results, and spit out one final, reliable verdict.
- Throw together a quick Streamlit UI so anyone can paste in a news headline and test it out themselves.

4. Dataset Used

To build the foundation for this project, I used the WELFake dataset, a collection of over 70,000 news articles. To get the data ready for my experiments, I processed it by:

- **Structuring the labels:** Using the dataset's straightforward categorization, where 1 flag fake news and 0 marks real news.
- **Merging the content:** Combining the headline and the main body text of each article to give the model the complete context of the story.
- **Cleaning and splitting:** Filtering the merged text to remove noise, and then dividing the dataset into an 80/20 split for training and testing.

Preparing the data this way ensured the machine learning models had a clean, comprehensive baseline to learn from.

5. Methodology

5.1 Data Preprocessing

I cleaned the text by:

- Removing URLs
- Lowercasing everything
- Removing extra spaces and special characters
- Merging title and body text

This prepared the dataset for training.

5.2 Model Training

I used **DistilBERT**, a smaller version of BERT, because it is fast and still performs well.
I fine-tuned it for binary classification using:

- Learning rate: 2e-5
- Optimizer: AdamW
- Epochs: 1
- Batch size: 8

The trained model was saved inside models/best_model/.

5.3 Evaluation

I evaluated the model on 14,427 test samples.

The results were:

- Accuracy: 97.93%
- Precision: 95.76%

- Recall: 99.04%
- F1 Score: 98.83%

Confusion Matrix:

	Predicted Real	Predicted False
Actual Real	6965	96
Actual False	64	7274

Overall, these results highlight how well the model performs, showing a strong level of accuracy with very few incorrect predictions.

6. Hybrid Online Verification System

Since a static model gets outdated quickly with breaking news, I built a second layer that verifies claims live using the Wikipedia, GNews, and Google Fact Check APIs.

The logic I set up is pretty straightforward:

- If Google Fact Check explicitly flags it as false, the final output is FAKE.
- If Wikipedia has a solid summary matching the claim, it's marked TRUE.
- If GNews pulls up recent articles backing it up, it's TRUE.
- If the AI thinks it's fake but the web has zero proof either way, it flags it as UNCERTAIN.
- If none of these triggers, it just defaults to the AI's original guess.

Ultimately, this hybrid setup makes the whole system way more reliable, especially when dealing with fresh or trending stories.

7. System Architecture

Here is how the whole pipeline actually works from start to finish:

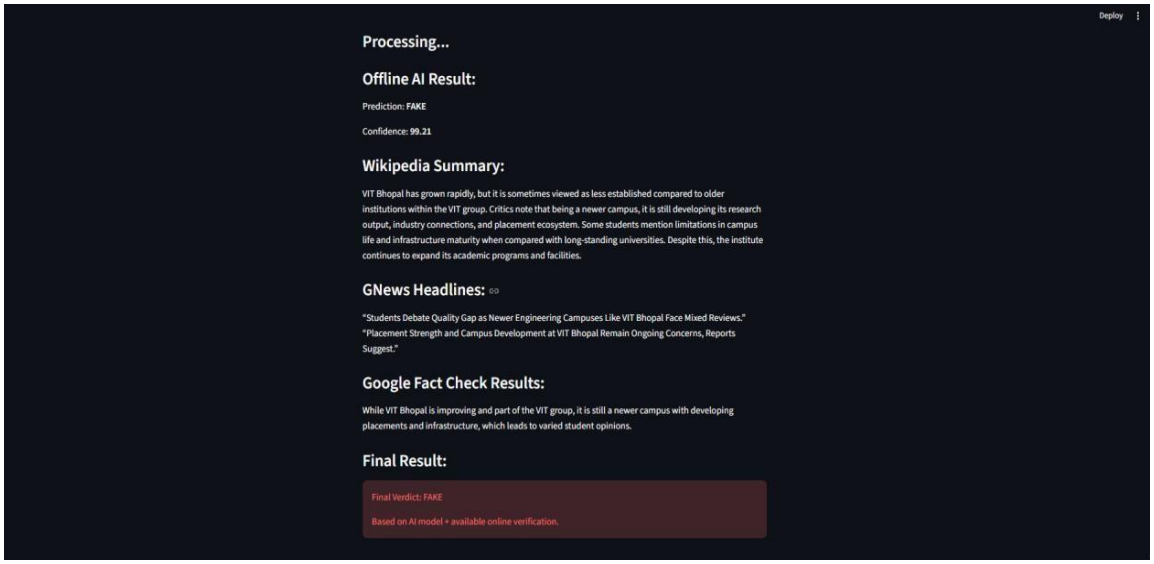
1. First, you just drop in the news article or headline you want to check.
2. The offline AI takes the first pass and makes a baseline guess on whether it's real or fake.
3. Right after that, the system pings the live web APIs to gather real-world proof.
4. The hybrid logic kicks in to weigh the AI's prediction against the live web data.
5. Finally, it spits out the final, verified verdict on the screen for you.

```
(venv) PS C:\FakeNews-main> streamlit run streamlit_app.py
>>

      Welcome to Streamlit!

      If you'd like to receive helpful onboarding emails, news, offers, promotions,
      and the occasional swag, please enter your email address below. Otherwise,
      leave this field blank.

      Email: abc@email,com

  You can find our privacy policy at https://streamlit.io/privacy-policy

  Summary:
  - This open source library collects usage statistics.
  - We cannot see and do not store information contained inside Streamlit apps,
    such as text, charts, images, etc.
  - Telemetry data is stored in servers in the United States.
  - If you'd like to opt out, add the following to %userprofile%/.streamlit/config.toml,
    creating that file if necessary:

    [browser]
    gatherUsageStats = false


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.152.210.145:8501
```

8. Streamlit User Interface

I put together a simple Streamlit interface where anyone can test the system. You just:

- Drop in whatever news headline or claim you want to check.
- Pick whether you want just the AI's guess or the full hybrid web check.
- View the actual proof it pulls in from Wikipedia, GNews, and Google Fact Check.



9. Results and Discussion

The model absolutely nailed the offline tests, hitting nearly 99% accuracy. But the hybrid setup is where it really shines in the real world, mostly because it doesn't just blindly trust the AI's first guess.

The biggest win here is that it drastically cuts down on false positives—even if the model gets confused, the live web check steps in to correct it. The only real catch is that because it relies on active internet searches and free APIs, you'll eventually hit rate limits if you try to run a massive number of checks at once.

10. Conclusion

To wrap things up, this project shows how powerful it can be to mix AI with real-time fact-checking. A standalone model is fine, but combining deep learning with live online searches makes the tool actually practical for the real world. I also loved putting together the Streamlit UI because it means anyone can test out a headline without needing to touch the code.

All in all, the project hit its marks and delivered really solid results, whether evaluating offline data or handling live news.

11. Future Improvements

If I continue working on this project, I would like to:

- Improve Wikipedia matching using semantic search
- Add more APIs for verification
- Deploy the system online using a cloud platform
- Extend support to image-based or video fake news
- Use a stronger model like DeBERTa or RoBERTa

12. References

- HuggingFace Transformers Library
- GNews API
- PyTorch Documentation
- Google Fact Check Tools API
- WELFake Dataset
- Wikipedia API